Cómo resolver un problema mediante recurrencias

Diez discos y una pregunta

Tienes diez discos en las Torres de Hanói. Debes llevarlos de la torre A a la C usando B como auxiliar. Solo puedes mover un disco a la vez y nunca colocar uno grande sobre uno pequeño. ¿Cómo puedes saber cuántos movimientos necesitarás sin jugar toda la partida?

Con un disco necesitas un movimiento. Con dos necesitas tres. Al añadir un tercer disco, ¿basta con añadir otro movimiento? No: primero debes liberar el disco mayor y después reconstruir la torre sobre él. El número de movimientos depende del trabajo necesario para trasladar los discos menores.

Encontrar una relación antes de calcular

Llama M(n) al mínimo de movimientos para n discos. Si logras expresar M(n) mediante M(n − 1), podrás usar la misma regla para tres, cuatro o diez discos. Esa relación es una recurrencia. Para usarla necesitas también un caso cuyo resultado ya conozcas: el caso base.

Empezaremos con el factorial de 50 y Fibonacci, donde es más fácil reconocer esta dependencia. Después volveremos a Hanói para responder la pregunta inicial. MergeSort permitirá analizar el trabajo por niveles y damas chinas mostrará cómo explorar distintas continuaciones de una jugada.

Qué debes distinguir al leer una ecuación

Símbolo	Qué representa
n!	El valor del factorial.
F(n)	Un número de Fibonacci.
M(n)	La cantidad de movimientos en Hanói.
T(n)	El tiempo o trabajo del algoritmo.

Estas cantidades no son intercambiables. Una fórmula puede decirte cuál es la respuesta y otra cuánto trabajo necesitas para obtenerla. El factorial crece muy rápidamente, pero eso no significa que el programa haga tantas operaciones como el valor del factorial.

En el análisis contaremos las operaciones básicas como de costo constante. Θ indica un orden de crecimiento ajustado; O indica una cota superior. Los códigos completos están en los archivos adjuntos; aquí veremos cómo cada instrucción se relaciona con el razonamiento.

1 Cómo calcular el factorial de 50

El factorial n! es el producto de los enteros desde 1 hasta n. Por ejemplo, 4! = 4 × 3 × 2 × 1 = 24. Para 50 tendríamos que multiplicar 50 × 49 × … × 2 × 1.

Si conocieras 49 factorial qué te faltaría hacer

Solo faltaría multiplicarlo por 50. Todo el producto restante ya está contenido en 49!. Así obtenemos una regla que sirve para cualquier entero n ≥ 1:

n! = n × (n − 1)!; caso base: 0! = 1.

Seguir un caso pequeño hasta completar la respuesta

Para calcular	Primero necesitamos	Al regresar obtenemos
3!	2!	3 × 2 = 6
2!	1!	2 × 1 = 2
1!	0!	1 × 1 = 1
0!	Ningún otro factorial	1 por definición

Lee la columna central hacia abajo: son los problemas pendientes. Después lee la última columna hacia arriba: son las respuestas que regresan. La llamada con n = 3 espera; no multiplica hasta recibir el resultado de n = 2.

El código sigue esa misma regla

```
def factorial_rec(n):
    if n == 0:
        return 1
    return n * factorial_rec(n - 1)
```

if n == 0 reconoce el caso base. return 1 entrega su respuesta. La última línea pide el factorial menor y luego multiplica por n. Ejecutar print(factorial_rec(50)) devuelve un entero exacto en Python; su aproximación es $3.04140932 \times 10^{64}$.

Cuánto trabajo realiza

Cada llamada hace una nueva llamada y una multiplicación. Si c es el trabajo adicional y d el costo del caso base:

T(n) = T(n − 1) + c; T(0) = d.

Para n = 3: T(3) = T(2) + c = T(1) + 2c = T(0) + 3c. En general T(n) = d + nc, de modo que T(n) = Θ(n). Hay n multiplicaciones; no n! multiplicaciones. La pila de llamadas ocupa Θ(n) espacio. Para enteros grandes, el costo real de multiplicarlos puede aumentar.

2 Qué número de Fibonacci corresponde a la generación 10

Usaremos F(0) = 0 y F(1) = 1. Cada valor siguiente es la suma de los dos anteriores:

F(n) = F(n − 1) + F(n − 2), para n ≥ 2.

Para F(10) necesitas F(9) y F(8). Pero esos valores también dependen de anteriores. Llegas a los casos base y completas las sumas:

n	0	1	2	3	4	5	6	7	8	9	10
F(n)	0	1	1	2	3	5	8	13	21	34	55

F(10) = F(9) + F(8) = 34 + 21 = 55.

```
def fibonacci_rec(n):
    if n <= 1:
        return n
    return fibonacci_rec(n - 1) + fibonacci_rec(n - 2)
```

Cuando n vale 0 o 1, return n devuelve el caso base. Para cualquier otro n, el programa calcula dos resultados y los suma. La regla del valor no añade una constante.

Qué se repite si calculas F de 4

F(4) pide F(3) y F(2). A su vez, F(3) pide F(2) y F(1). Por tanto F(2) se calcula dos veces, aunque siempre devuelve 1. En índices mayores esa repetición se multiplica.

La recurrencia del tiempo responde otra pregunta

T(n) = T(n − 1) + T(n − 2) + c.

El primer término cuenta el trabajo de la primera llamada; el segundo, el de la otra; c cuenta comprobar, sumar y devolver. Esta constante pertenece al trabajo del programa, no al valor F(n).

El número de llamadas crece exponencialmente: para F(10) hay 177 llamadas, aunque el resultado sea 55. Una cota temporal válida es $O(2^n)$. La cota ajustada es $\Theta(\varphi^n)$, donde $\varphi \approx 1.618$; no necesitamos esa fórmula para reconocer el trabajo repetido. La profundidad de las llamadas, y por tanto la pila, es $\Theta(n)$.

3 Cómo evitar calcular dos veces la misma respuesta

Si ya calculaste F(8), ¿por qué volver a resolverlo cuando otra llamada lo necesita? Podemos guardarlo. Cuando se solicite de nuevo, recuperamos su respuesta. Esta técnica se llama memoización.

Qué guardamos y cuándo lo usamos

memoria es un diccionario: asocia cada índice con su número de Fibonacci. Al iniciar ya conoce 0 y 1. Para un índice nuevo calcula el resultado y lo guarda; para uno conocido lo devuelve directamente.

```
def fibonacci_memo(n, memoria=None):
    if memoria is None:
        memoria = {0: 0, 1: 1}
    if n not in memoria:
        memoria[n] = (fibonacci_memo(n - 1, memoria)
                      + fibonacci_memo(n - 2, memoria))
    return memoria[n]
```

La primera condición crea una memoria nueva cuando comienza la consulta. La segunda pregunta si falta n. Las llamadas internas reciben el mismo diccionario: así pueden reutilizar lo que otra llamada ya calculó.

Seguir la consulta F de 4

Momento	Qué ocurre
Inicio	La memoria conoce F(0) = 0 y F(1) = 1.
Primera consulta de F(2)	Calcula 1 + 0 y guarda F(2) = 1.
Consulta de F(3)	Usa F(2) y F(1); guarda F(3) = 2.
Segunda consulta de F(2)	Recupera 1 sin volver a calcularlo.
Consulta de F(4)	Suma 2 + 1 y guarda F(4) = 3.

La respuesta se conserva y el trabajo cambia

fibonacci_memo(10) también devuelve 55. Cada índice de 2 a n se calcula una sola vez: hay $n - 1$ sumas nuevas. Con consultas de costo promedio constante y sumas de costo constante, el tiempo es $\Theta(n)$. La memoria y la pila ocupan $\Theta(n)$ espacio.

No podemos seguir contando ambas ramas como cálculos completos cada vez: una consulta puede encontrar una respuesta ya guardada. Para analizar el costo debemos mirar lo que hace esta implementación.

4 Cómo trasladar los discos de Hanói

Volvamos a los diez discos. El disco mayor está debajo de todos. ¿Qué necesitas hacer antes de moverlo a C? Debes llevar los otros nueve a B. ¿Y después? Llevar esos nueve de B a C, colocándolos sobre el mayor.

Tres etapas que sirven para cualquier n

Etapa	Acción	Movimientos
1	Llevar n − 1 discos de A a B usando C.	M(n − 1)
2	Mover el disco mayor de A a C.	1
3	Llevar n − 1 discos de B a C usando A.	M(n − 1)

M(n) = M(n − 1) + 1 + M(n − 1) = 2M(n − 1) + 1.

El caso base M(0) = 0 significa que no hay discos que mover. Por eso M(1) = 2 × 0 + 1 = 1. Para dos discos M(2) = 2 × 1 + 1 = 3; para tres, M(3) = 2 × 3 + 1 = 7.

Cómo cambian los papeles de las torres en el código

```
def hanoi(n, origen, auxiliar, destino, mover):
    if n == 0:
        return 0
    antes = hanoi(n - 1, origen, destino, auxiliar, mover)
    mover(n, origen, destino)
    despues = hanoi(n - 1, auxiliar, origen, destino, mover)
    return antes + 1 + despues
```

La primera llamada termina en la auxiliar: sus argumentos cambian los papeles de B y C. Luego mover ejecuta el movimiento del disco mayor. La segunda llamada empieza en B y termina en C. mover es una función que recibe disco, origen y destino; puede imprimir o ejecutar ese movimiento.

Con tres discos, la secuencia es: disco 1 de A a C; disco 2 de A a B; disco 1 de C a B; disco 3 de A a C; disco 1 de B a A; disco 2 de B a C; disco 1 de A a C. Los pasos 1 a 3 liberan el mayor; el paso 4 lo traslada; los pasos 5 a 7 completan la torre.

El código devuelve la cantidad de movimientos: suma antes + 1 + despues. El archivo Python incluye mostrar para imprimirlos y permite comprobar que diez discos requieren 1023 movimientos. Ahora falta justificar ese número mediante la recurrencia.

5 Obtener una fórmula para los movimientos de Hanói

Desarrollar la regla hasta llegar al caso base

Partimos de $M(n) = 2M(n - 1) + 1$. Para dejar de depender de $M(n - 1)$, lo sustituimos por $2M(n - 2) + 1$. Hay que multiplicar toda esa expresión por 2:

$M(n) = 2[2M(n - 2) + 1] + 1 = 4M(n - 2) + 3.$

Si sustituimos nuevamente: $M(n) = 4[2M(n - 3) + 1] + 3 = 8M(n - 3) + 7$. Los coeficientes son 2, 4, 8; los términos adicionales son 1, 3, 7.

Después de k pasos: $M(n) = 2^k M(n - k) + (2^k - 1).$

Para llegar a $M(0)$, elegimos $k = n$. Como $M(0) = 0$, obtenemos:

$M(n) = 2^n - 1$; por tanto $M(10) = 1024 - 1 = 1023.$

Por qué el patrón necesita una demostración

El desarrollo sugiere la fórmula. El método de sustitución la verifica mediante inducción. Primero revisamos el caso base: $2^0 - 1 = 0$. Después suponemos que funciona para $n - 1$ y comprobamos el siguiente caso:

$M(n) = 2(2^{n-1} - 1) + 1 = 2^n - 2 + 1 = 2^n - 1.$

La fórmula cumple tanto el caso base como la regla recursiva. Esto justifica su uso para cualquier entero $n \geq 0$, no solo para los primeros ejemplos.

Por qué no podemos hacerlo con menos movimientos

Antes de mover el mayor a su destino, los menores deben estar en la auxiliar. Después deben ir al destino. Ambos traslados son obligatorios; la estrategia los resuelve con el mínimo correspondiente a $n - 1$ discos. Así alcanza el mínimo total.

Del número de movimientos al tiempo

Con mover de costo constante, $T(n) = 2T(n - 1) + c$ y el tiempo es $\Theta(2^n)$. La pila ocupa $\Theta(n)$ espacio. $M(n)$ cuenta movimientos exactos; $T(n)$ cuenta también comprobaciones y llamadas. Hanói se descompone recursivamente, pero sus subproblemas disminuyen en uno y se realizan en secuencia.

6 Cómo ordenar una lista con divide y vencerás

Quieres ordenar [8, 3, 5, 1]. Si obtuvieras dos mitades ya ordenadas, ¿podrías unirlas sin volver a ordenar todo? Sí: compara sus primeros elementos disponibles y elige el menor.

Dividir resolver y combinar

Divide en [8, 3] y [5, 1]. Cada mitad se divide otra vez hasta listas de un elemento, que ya están ordenadas. Al combinar obtienes [3, 8] y [1, 5]. Para unirlas eliges 1, después 3, después 5 y finalmente 8. El resultado es [1, 3, 5, 8].

```
def merge_sort(datos):
    if len(datos) <= 1:
        return datos.copy()
    mitad = len(datos) // 2
    izquierda = merge_sort(datos[:mitad])
    derecha = merge_sort(datos[mitad:])
    resultado = []
    i = j = 0
    while i < len(izquierda) and j < len(derecha):
        if izquierda[i] <= derecha[j]:
            resultado.append(izquierda[i])
            i += 1
        else:
            resultado.append(derecha[j])
            j += 1
    resultado.extend(izquierda[i:])
    resultado.extend(derecha[j:])
    return resultado
```

El caso len(datos) <= 1 detiene la división. Las dos llamadas ordenan las mitades. Los índices i y j señalan el siguiente elemento disponible de cada mitad. El ciclo añade el menor y avanza su índice; extend añade los elementos restantes cuando una mitad se agota.

Cómo construir su recurrencia temporal

Hay dos llamadas con aproximadamente la mitad de los datos. Combinar examina todos los elementos: cuesta Θ(n). Copiar las mitades también es lineal y queda incluido en ese término.

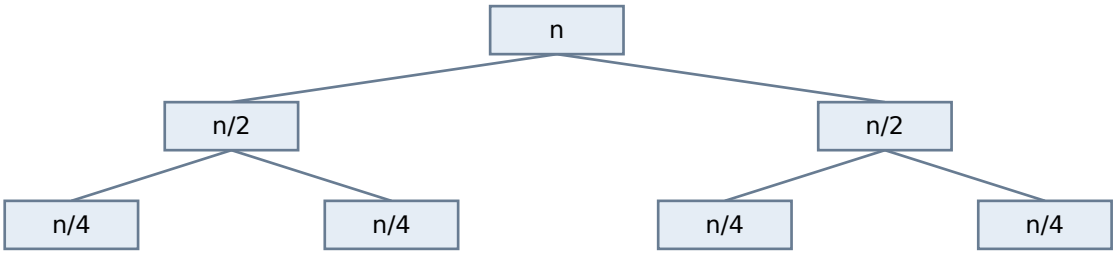
T(n) = 2T(n/2) + Θ(n), con caso base constante.

Esta es la estructura típica de divide y vencerás: dos subproblemas menores y una combinación de sus soluciones. Para n impar las mitades tienen tamaños piso(n/2) y techo(n/2); simplificaremos el desarrollo suponiendo n potencia de dos.

7 Contar el trabajo por niveles de MergeSort

¿Por qué ordenar n elementos no cuesta simplemente Θ(n)? La combinación es lineal, pero se repite en varios niveles. El árbol de recursión permite sumar ese trabajo sin contar dos veces las llamadas.

Paso 1 Representar los subproblemas



Paso 2 Calcular el costo total de cada nivel

Cada recuadro muestra el tamaño del subproblema. Con un costo local cn, el total de cada nivel es:

Nivel	Problemas	Costo de cada uno	Total
0	1	cn	cn
1	2	cn/2	cn
2	4	cn/4	cn

La cantidad de problemas se duplica, pero su tamaño se divide entre dos. Por eso el trabajo total del nivel sigue siendo cn.

Paso 3 Encontrar la profundidad

Para n = 8, los tamaños son 8, 4, 2 y 1. Hay tres niveles de combinación y un nivel de casos base. En general llegamos a tamaño uno cuando:

$$n/2^j = 1 \Rightarrow j = \log_2 n.$$

Paso 4 Sumar los niveles y las hojas

Sumamos $\log_2 n$ niveles de costo cn y n hojas de costo d:

$$T(n) = cn \log_2 n + dn = \Theta(n \log n).$$

Para ocho elementos, el modelo da $3 \times 8c + 8d = 24c + 8d$. La expresión es exacta para este modelo simplificado; el código real conserva el mismo orden aunque sus constantes cambien. La memoria auxiliar máxima es Θ(n), incluida la salida; la profundidad de la pila es Θ(log n).

8 Resolver la misma recurrencia por sustitución

El árbol nos permitió descubrir una posible solución. Ahora vamos a comprobarla mediante otro método. Mantendremos el mismo modelo de MergeSort, para comparar los procedimientos sin cambiar el problema:

$T(n) = 2T(n/2) + cn; T(1) = d; c,d > 0.$

Suponemos n potencia de dos. c representa el costo por elemento en el trabajo local y d el costo de una hoja. Estas constantes se conservan en los tres métodos.

Paso 1 Proponer una solución

El árbol sugiere $\log_2 n$ niveles de combinación y n hojas. Proponemos $H(n) = cn \log_2 n + dn$. Es una candidata; todavía debemos demostrar que satisface la recurrencia.

Paso 2 Comprobar el caso base

$H(1) = c \times 1 \times \log_2 1 + d \times 1 = 0 + d = T(1).$

La fórmula funciona para $n = 1$ porque $\log_2 1 = 0$. No debemos omitir este caso: el término dn conserva precisamente el costo de las hojas.

Paso 3 Suponer que funciona para el problema menor

$T(n/2) = c(n/2) \log_2(n/2) + d(n/2).$

Esta es la hipótesis de inducción. Suponemos que la fórmula vale para la mitad de la entrada y preguntamos si entonces también vale para n.

Paso 4 Sustituir toda la expresión y simplificar

$T(n) = 2[c(n/2) \log_2(n/2) + d(n/2)] + cn.$

El 2 multiplica ambos términos dentro del corchete. Como $2 \times (n/2) = n$, queda:

$T(n) = cn \log_2(n/2) + dn + cn.$

Usamos $\log_2(n/2) = \log_2 n - \log_2 2 = \log_2 n - 1$. Entonces:

$T(n) = cn(\log_2 n - 1) + dn + cn.$

$T(n) = cn \log_2 n - cn + dn + cn = cn \log_2 n + dn.$

Paso 5 Interpretar lo que demostramos

El caso base y el paso inductivo confirman la candidata. La fórmula es exacta para este modelo y, como c,d son positivos, su término de mayor orden da $T(n) = \Theta(n \log n)$. Sustitución no consiste solo en adivinar: consiste en justificar la solución propuesta.

9 Resolver la misma recurrencia con el Teorema Maestro

Partimos otra vez de T(n) = 2T(n/2) + cn, con T(1) = d y c,d > 0. El Teorema Maestro busca su orden de crecimiento; no proporciona por sí solo los coeficientes de la fórmula exacta.

Paso 1 Verificar la estructura

Forma requerida: T(n) = aT(n/b) + f(n).

a y b deben ser constantes, con a ≥ 1 y b > 1. MergeSort tiene dos llamadas que reducen el tamaño a la mitad, por lo que cumple esta estructura.

Elemento	En nuestro problema	Interpretación
a	2	Dos llamadas recursivas.
b	2	Cada entrada se divide entre dos.
f(n)	cn	Trabajo fuera de las llamadas.

Paso 2 Calcular el término de comparación

El teorema compara f(n) con g(n), definido como n elevado a log_b(a). Sustituimos primero la base b = 2 y el argumento a = 2:

$\log_b(a) = \log_2 2 = 1$; por tanto $g(n) = n^1 = n$.

Paso 3 Comparar los órdenes

Como c es una constante positiva, f(n) = cn tiene orden Θ(n). El término g(n) también es n. Ninguno tiene un orden mayor: estamos en el caso de equilibrio.

Paso 4 Aplicar el caso correspondiente

Si f(n) = Θ(g(n)), entonces T(n) = Θ(g(n) log n).

Sustituimos g(n) = n y obtenemos T(n) = Θ(n log n). El resultado coincide con el árbol y la sustitución, pero aquí usamos una regla ya demostrada.

Por qué no basta con memorizar la conclusión

Si cambian el número de llamadas, el tamaño de los subproblemas o f(n), cambia la comparación. Antes de usar el teorema debemos identificar los tres componentes. Hanói, factorial y Fibonacci directo no cumplen su forma estándar.

Los otros casos exigen una diferencia polinómica de órdenes: f(n) = O(n elevado a [log_b(a) − ε]) da Θ(g(n)); f(n) = Ω(n elevado a [log_b(a) + ε]) da Θ(f(n)) si además a f(n/b) ≤ q f(n), con q < 1. En ambos, ε > 0. Esta última condición se llama regularidad. Para MergeSort basta el caso de equilibrio desarrollado arriba.

10 Contrastar los tres métodos sin cambiar el problema

Problema común: $T(n) = 2T(n/2) + cn$; $T(1) = d$.

Los tres métodos analizan la misma cantidad y usan los mismos supuestos. Sus resultados coinciden, pero hacen trabajos distintos. La tabla muestra qué aporta cada uno.

Criterio	Sustitución	Árbol de recursión	Teorema Maestro
Por dónde comienzas	Una fórmula o cota candidata.	Las llamadas y sus costos locales.	La forma $aT(n/b) + f(n)$.
Qué calculas	La candidata dentro de la regla.	Costo por nivel, profundidad y hojas.	a, b y el orden de f(n) frente a g(n).
Qué justifica la solución	Caso base y paso inductivo.	Suma de todo el trabajo del árbol.	Cumplimiento de un caso del teorema.
Resultado aquí	$cn \log_2 n + dn$, demostrado.	$cn \log_2 n + dn$, al sumar niveles y hojas.	$\Theta(n \log n)$, como orden temporal.
Dificultad habitual	Proponer una candidata y cerrar la inducción.	Contar correctamente niveles y casos base.	Verificar que la regla se pueda aplicar.

Una comprobación numérica de la fórmula

Para comprobar unos valores tomamos $c = d = 1$. La recurrencia es $T(n) = 2T(n/2) + n$ y $T(1) = 1$. La fórmula de los dos primeros métodos es $T(n) = n \log_2 n + n$.

n	Desde la recurrencia	Desde la fórmula
1	Caso base: 1	$1 \times 0 + 1 = 1$
2	$2 \times 1 + 2 = 4$	$2 \times 1 + 2 = 4$
4	$2 \times 4 + 4 = 12$	$4 \times 2 + 4 = 12$
8	$2 \times 12 + 8 = 32$	$8 \times 3 + 8 = 32$

Comprobar números permite detectar errores, pero no sustituye la demostración para cualquier n. La inducción hizo esa demostración; el árbol explicó de dónde salió la candidata; el teorema confirmó su orden.

Cómo elegir una ruta de solución

Si no reconoces el crecimiento, empieza por expandir o dibujar las llamadas. Si tienes una candidata, demuéstrala por sustitución. Si reconoces la forma del Teorema Maestro, verifica sus condiciones. Puedes usar más de un método: descubrir con uno y comprobar con otro.

El desarrollo repetido de Hanói ayudó a descubrir $2^n - 1$; la inducción lo justificó. No confundas ese desarrollo con el método de sustitución basado en una candidata. Tampoco confundas una fórmula exacta con una cota asintótica.

11 Cómo buscar una solución en damas chinas

Tienes diez fichas en la zona inicial. ¿Cómo puede un programa encontrar una secuencia de jugadas que las lleve a la zona opuesta sin explorar para siempre? Consideraremos una versión individual: solo tus fichas están en el tablero.

Qué describe un estado y qué cuenta como jugada

Un estado S indica dónde están tus diez fichas. La meta es ocupar las diez casillas del triángulo opuesto. Una jugada es un desplazamiento a una casilla vecina vacía o una secuencia de saltos legales; las fichas saltadas permanecen en el tablero. La búsqueda dispone de un límite d de jugadas.

Qué sucede después de una primera jugada

Supón que desde S puedes llegar a A o a B. Con dos jugadas disponibles, probar A significa preguntar si desde A puedes alcanzar la meta con una jugada restante. Si no funciona, regresas y pruebas B. El tablero conserva su tamaño; lo que disminuye es el número de jugadas que puedes explorar.

R(S,d): ¿existe una solución desde S con como máximo d jugadas?

Situación	Respuesta
S ya es la meta	Verdadero, incluso si d = 0.
S no es la meta y d = 0	Falso: no quedan jugadas.
S no es la meta y d > 0	Probar R(S′,d − 1) para cada siguiente estado legal. Basta una respuesta verdadera.

Si J(S) contiene los estados obtenidos mediante una jugada legal, la tercera regla se escribe:

R(S,d) = OR de R(S′,d − 1), para S′ en J(S).

OR significa “alguna alternativa funciona”. Si ninguna llega a la meta, o no hay jugadas disponibles, la respuesta es falsa. Volver para probar otra alternativa es búsqueda recursiva con retroceso; evitar estados que ya están en la ruta elimina vueltas innecesarias.

Qué demuestra la respuesta de la búsqueda

Una ruta encontrada demuestra que existe una solución dentro de d jugadas. No demuestra que sea la más corta. No encontrarla solo descarta ese límite. Si descartas todos los límites menores y encuentras una ruta de longitud d, entonces has demostrado que d es el mínimo.

Con hasta b continuaciones por estado hay como máximo $1 + b + \ldots + b^d$ nodos; para $b > 1$ es $O(b^d)$. El tiempo también depende de generar y comparar estados. resolver_damas, en el archivo Python, implementa el motor recursivo: requiere proporcionar es_meta y sucesores con las reglas del tablero. No incluye un juego completo.